How I’m Using Claude Skills To Blow Up My Claude Code Workflows

Joe Njenga Follow

Claude Skills — Featured Image / By Author

Claude Code just got better with the new Claude Skills, a way to teach Claude your workflows once and have it execute them every single time.

Sounds unbelievable, but after reading this article and watching my demos, you will understand the leverage Claude Skills bring to Claude Code workflows.

If you have no idea what Claude Skills are, you will learn everything you should know and even practice from my in-depth breakdown of the problem they solve and how you can start using them today.

Quick Note: If you are not a premium member, you can read the story here. Consider supporting me by following me on Medium and my YouTube channel to learn more about Claude Code.

I've been using Claude Code for months, and teaching you here on Medium how to get the best out of every new feature, but there's been this nagging problem that's core to every need of automation — repetition.

Every single time you start a new project, you find yourself explaining the same things to Claude over and over again:

> "Add TypeScript with these specific tsconfig settings."
>
> "Set up my preferred ESLint configuration."
>
> "Create a React component following my team's naming conventions."

This is a problem for all of us, and we have tried to fix this issue using different Claude Code features that we discussed earlier, like my tutorials on Claude Code slash commands, Claude Code hooks, and other tutorials here.

Most developers spend the first 30 minutes of every project re-teaching Claude their workflows, preferences, and standards.

If this is you, then Claude Skills can quickly help you save that valuable time by automating the knowledge transfer process — turning your repetitive explanations into reusable, executable instructions that Claude loads automatically whenever needed.

Now, you might be thinking:

> "Wait, isn't this what Model Context Protocol (MCP) solves?"

Not quite.

MCP is powerful for giving Claude access to external data sources and tools, such as connecting to databases, APIs, or file systems, as I have covered extensively and shared a list of the best MCP servers here.

> But Skills are different; they are about teaching Claude how to DO things.

They're executable knowledge packages that tell Claude: "When the user asks for X, here's how we do it in our organization."

> MCP connects Claude to your data; Skills teach Claude your craft.

After testing and discovering what Claude Skills can do, I realized I'd been working with Claude the hard way.

Let me show you how I'm using this feature to blow up my workflow.

What Are Claude Skills?

Claude Skills are portable instruction packages that teach Claude how to perform specialized tasks your way.

Let me break this down without the technical jargon.

A Skill is simply a folder that contains:

- SKILL.md — A markdown file with your instructions, patterns, and best practices
- Supporting resources — Scripts, templates, configuration files, or reference code
- Execution logic — Optional code for tasks that need to work the same way every time

I like to think of Skills as recipe cards for Claude.

> Just like you'd give someone a recipe to bake your grandmother's perfect chocolate cake Skills gives Claude step-by-step instructions for doing specific tasks exactly how you want them done.

Here's what a typical Skill looks like in your file system:

```
~/.claude/skills/
└── darkmode-toggle/
    ├── SKILL.md
    ├── templates/
    │   ├── toggle-button.html
    │   └── theme-styles.css
    └── scripts/
        └── theme-handler.js
```

In this example, the darkmode-toggle Skill contains everything Claude needs to add a professional dark mode toggle to any website—the instructions in SKILL.md, plus the actual template files it can reference or use.

The beauty of Skills is that they work everywhere: Claude.ai, Claude Code, and the API.

You build the Skill once, and it follows you across every Claude product you use.

> Skills are also composable, meaning Claude can automatically identify which skills are needed for a task and use multiple skills together seamlessly.

But here's what makes Skills powerful: Claude only loads them when they're relevant.

You don't manually select skills or tell Claude which one to use. Claude scans your available skills, identifies what's needed for your current task, and loads only the minimum required information.

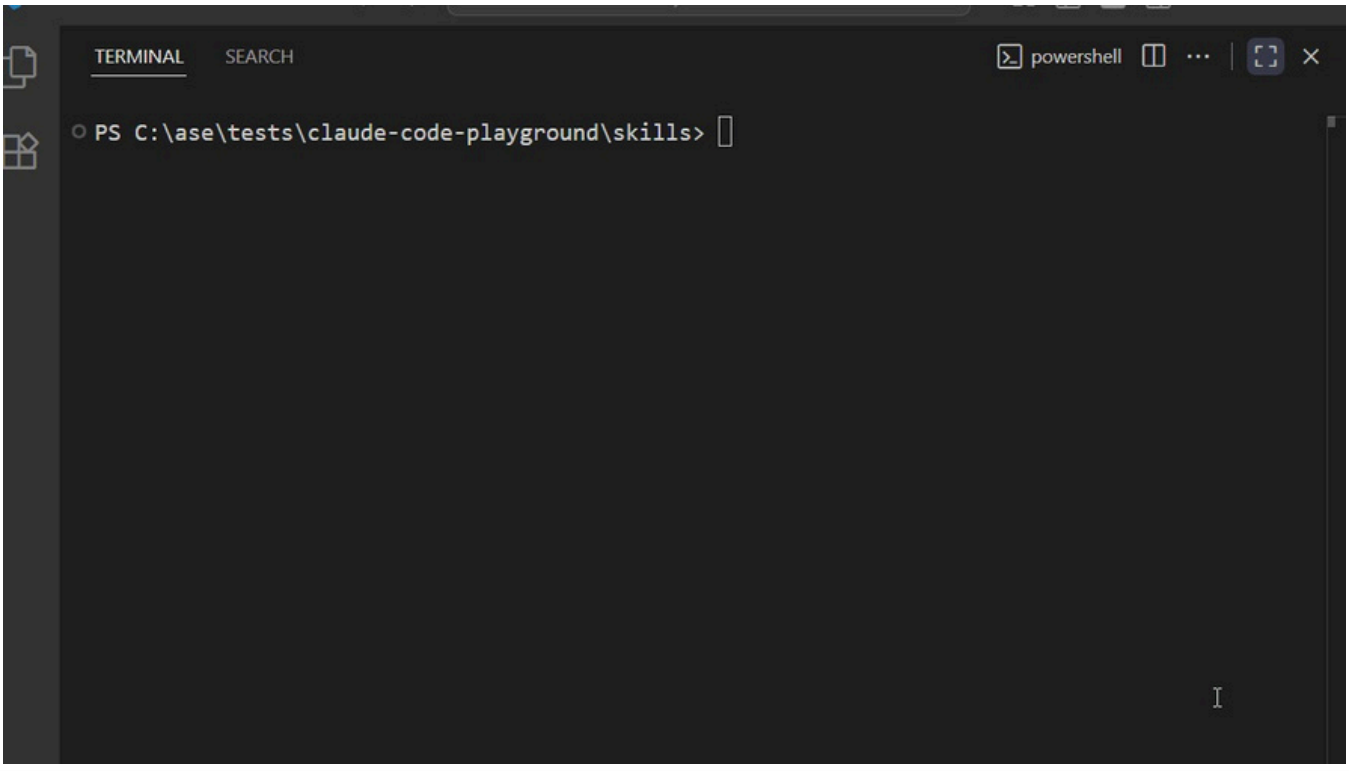
This keeps Claude fast while giving it access to your specialized expertise when needed.

Now let me show you how this works in practice.

How Claude Skills Work: A Real-World Example

To understand how skills work, see how I'm using this feature in a real project.

Right now, I have VS Code open with a basic web project: index.html, styles.css, and app.js — a blank slate.



The Problem I Want to Solve

What if I wanted to add a professional dark mode toggle to this website?

> Not just any dark mode but one that follows my specific implementation standards: smooth transitions, localStorage persistence, semantic HTML, and accessibility compliance.

Before Claude Skills, I'd open Claude Code and type out a detailed prompt explaining every single requirement:

```
"Add a dark mode toggle button in the top-right corner with a moon/sun emoji.
Use CSS custom properties for theming. Implement localStorage to persist the
preference. Add smooth transitions.
Make sure it's accessible with proper ARIA labels..."
```

And even then, Claude might miss a detail or implement it in a different way than I wanted.

Now the Claude Skills feature allows me to guide Claude Code in my specific way of creating this feature.

So where do we begin?

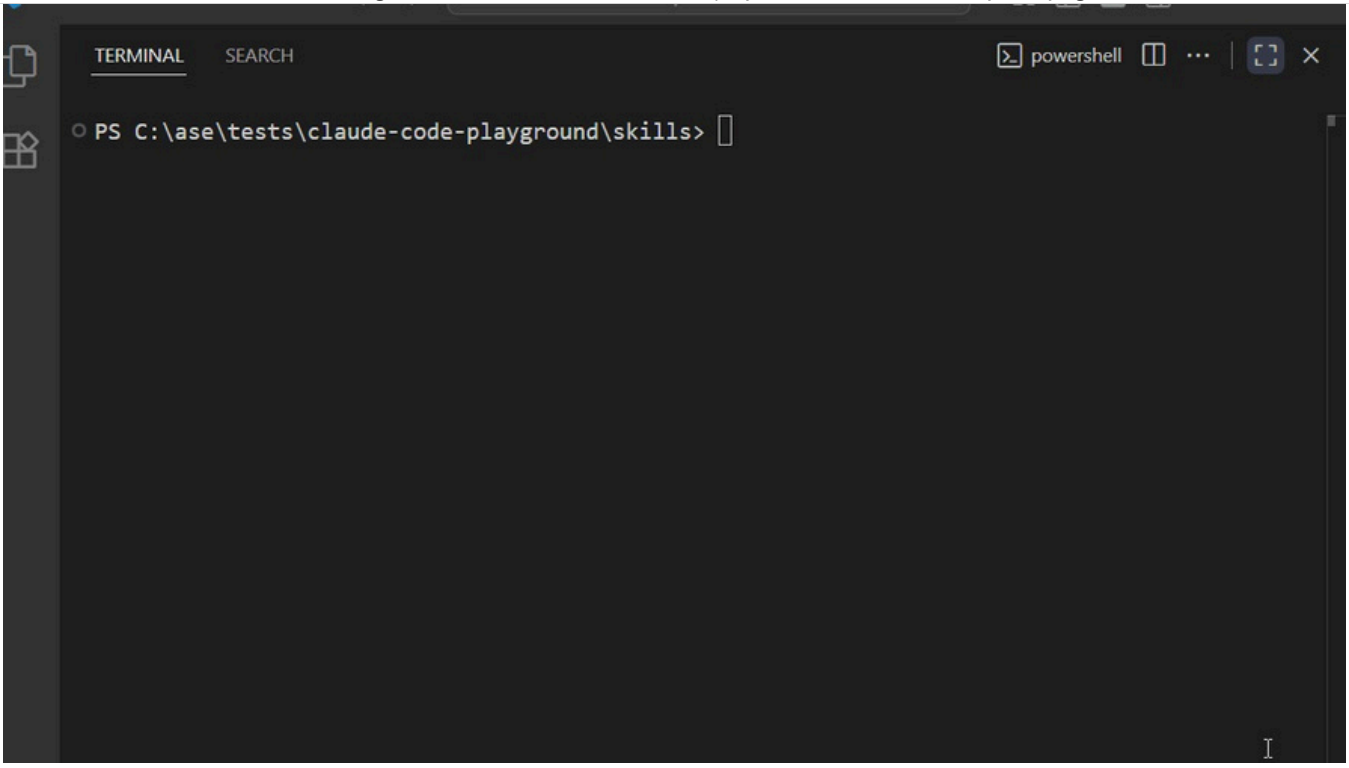
Creating My First Skill

Here's how I turned this into a reusable Claude Skill that I'll never have to explain again.

Step 1: Set up the Skill structure

I opened my terminal and created the Skill folder:

```
mkdir -p ~/.claude/skills/darkmode-toggle
cd ~/.claude/skills/darkmode-toggle
```



Step 2: Write the SKILL.md file

This is where the magic happens. I documented my exact implementation pattern in a file called SKILL.md.

I will open the new file in NodePad and paste this Skill code.

```
notepad SKILL.md
```

This will:

1. Open Notepad
2. Ask if you want to create a new file (click “Yes”)
3. Then paste this content:

```
---
name: Dark Mode Toggle
description: Add professional dark mode toggle to HTML websites with smooth tra
version: 1.0.0
---
```

```
# Dark Mode Toggle Skill

## Overview
This Skill provides a standardized implementation for adding dark mode function

## When to Use This Skill
- User asks to "add dark mode"
- User wants a "theme toggle" or "light/dark switch"
- User mentions making the site "dark mode compatible"

## Implementation Pattern

### HTML Structure
Add this toggle button to the body element:
```html
<button id="theme-toggle" aria-label="Toggle dark mode">🌙</button>
```

CSS Variables and Theming
Implement using CSS custom properties:
```css
:root {
 --bg-color: #ffffff;
 --text-color: #000000;
 --card-bg: #f5f5f5;
}

[data-theme="dark"] {
 --bg-color: #1a1a1a;
 --text-color: #ffffff;
 --card-bg: #2d2d2d;
}

body {
background-color: var(--bg-color);
color: var(--text-color);
 transition: background-color 0.3s ease, color 0.3s ease;
}

#theme-toggle {
 position: fixed;
 top: 20px;
 right: 20px;
 padding: 10px 15px;
 border: none;
 border-radius: 50%;
 background: var(--card-bg);
 cursor: pointer;
 font-size: 20px;
 box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
 transition: transform 0.2s ease;
}
```
```

```
#theme-toggle:hover {
 transform: scale(1.1);
}
```

JavaScript Logic

Implement with localStorage persistence:

```javascript
const themeToggle = document.getElementById('theme-toggle');
const currentTheme = localStorage.getItem('theme') || 'light';

// Set initial theme
document.documentElement.setAttribute('data-theme', currentTheme);
themeToggle.textContent = currentTheme === 'light' ? '🌙' : '☀️';

// Toggle functionality
themeToggle.addEventListener('click', () => {
 const theme = document.documentElement.getAttribute('data-theme');
 const newTheme = theme === 'light' ? 'dark' : 'light';


   document.documentElement.setAttribute('data-theme', newTheme);
 localStorage.setItem('theme', newTheme);
 themeToggle.textContent = newTheme === 'light' ? '🌙' : '☀️';
});
```

Quality Checklist

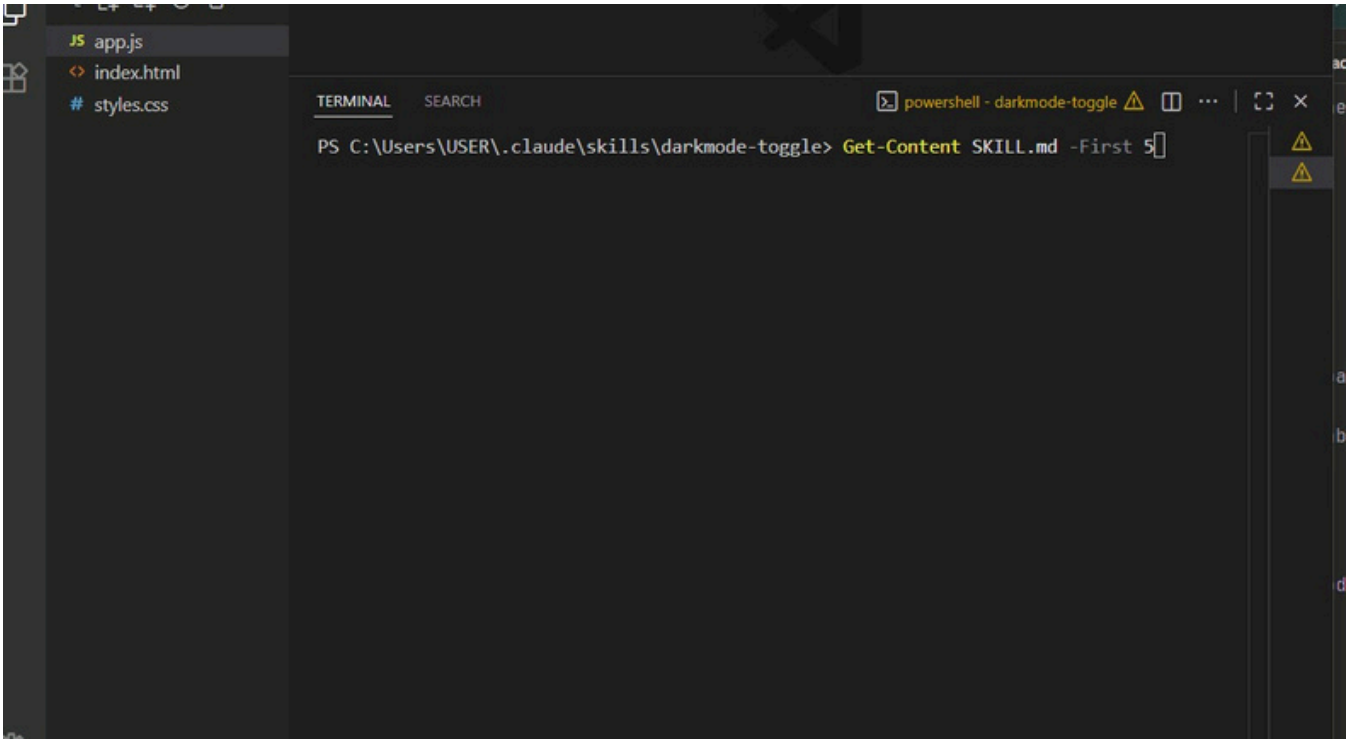
- Toggle button visible in top-right corner
- Smooth transition between themes (0.3s)
- Theme preference persists across page reloads
- Emoji icon updates based on current theme
- All existing elements adapt to theme variables
- Accessible with keyboard navigation

Save (Ctrl+S) and close. I saved this file in my ~/.claude/skills/darkmode-toggle/ directory.

Then verify with:

```
Get-Content SKILL.md
```

I have added the Skill and confirmed, as you can see in the demo above, and it's now ready.

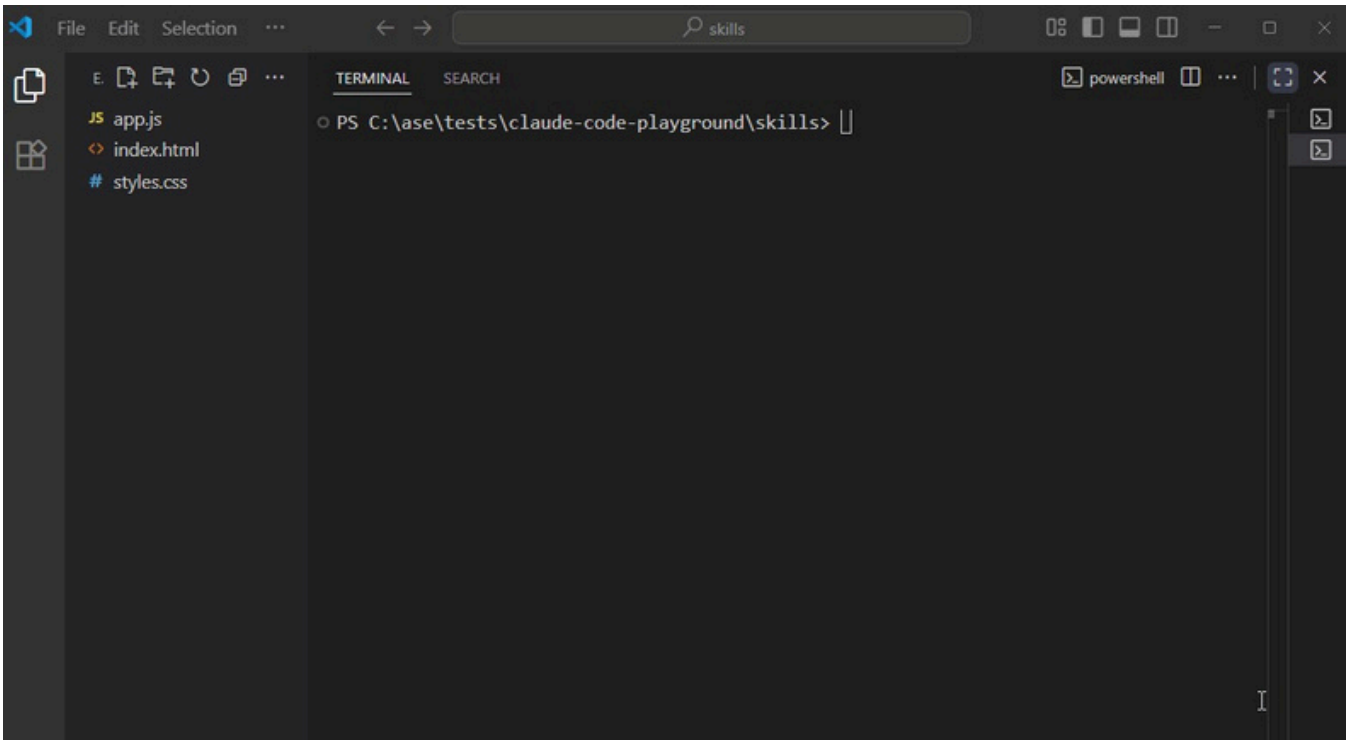


Step 3: Using the Skill

Now comes the automation part.

I go back to my VS Code project with those three files (index.html, styles.css, app.js), open Claude Code, and simply type:

"Add dark mode to this website"



Here’s what happens behind the scenes:

Skill Discovery — Claude scans my ~/.claude/skills/ folder and identifies that my "darkmode-toggle" skill is relevant to this request

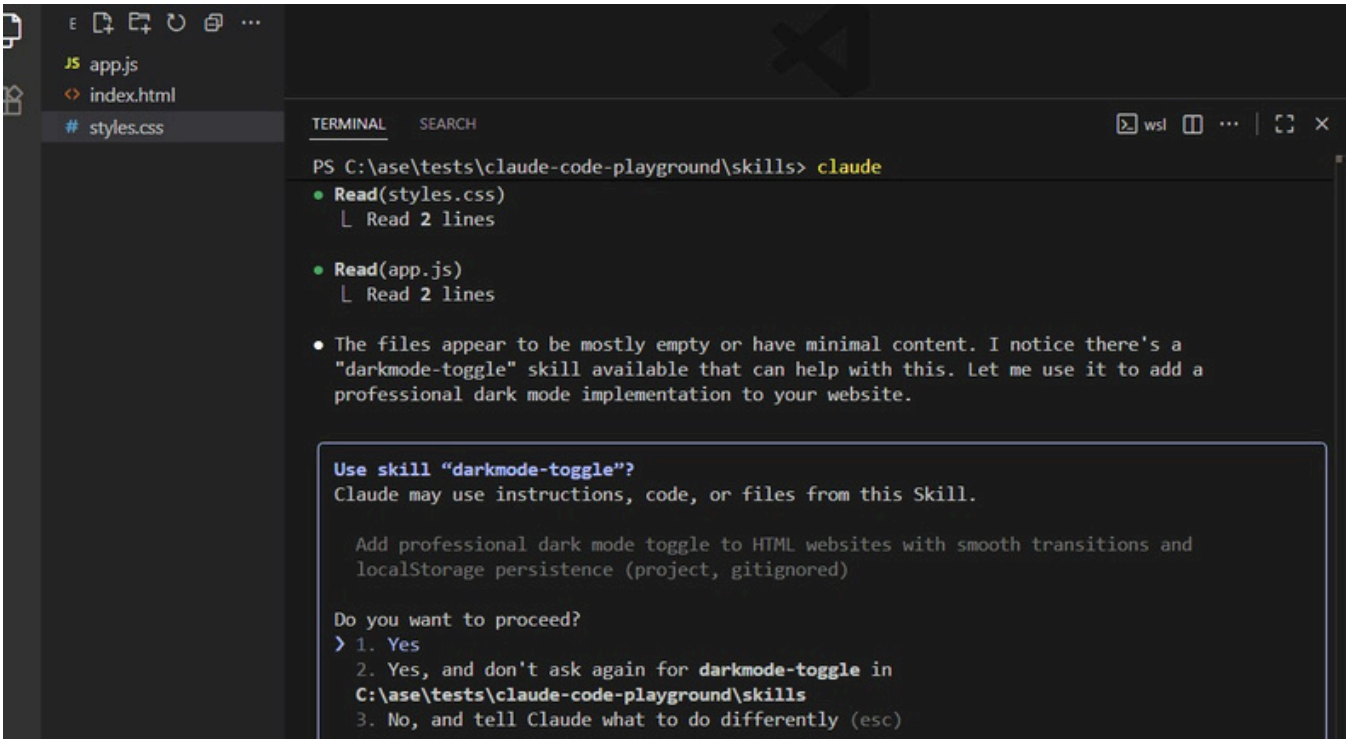
Skill Loading — Claude loads the SKILL.md file and reads my implementation pattern

Intelligent Application — Claude analyzes my existing files and applies the skill’s pattern:

- Adds the toggle button to my HTML
- Integrates the CSS variables into my existing styles without breaking anything
- Implements the JavaScript logic in my app.js file

Execution — Within seconds, my website now has a fully functional dark mode toggle that matches my exact specifications

How Claude Code Detects Required Skills

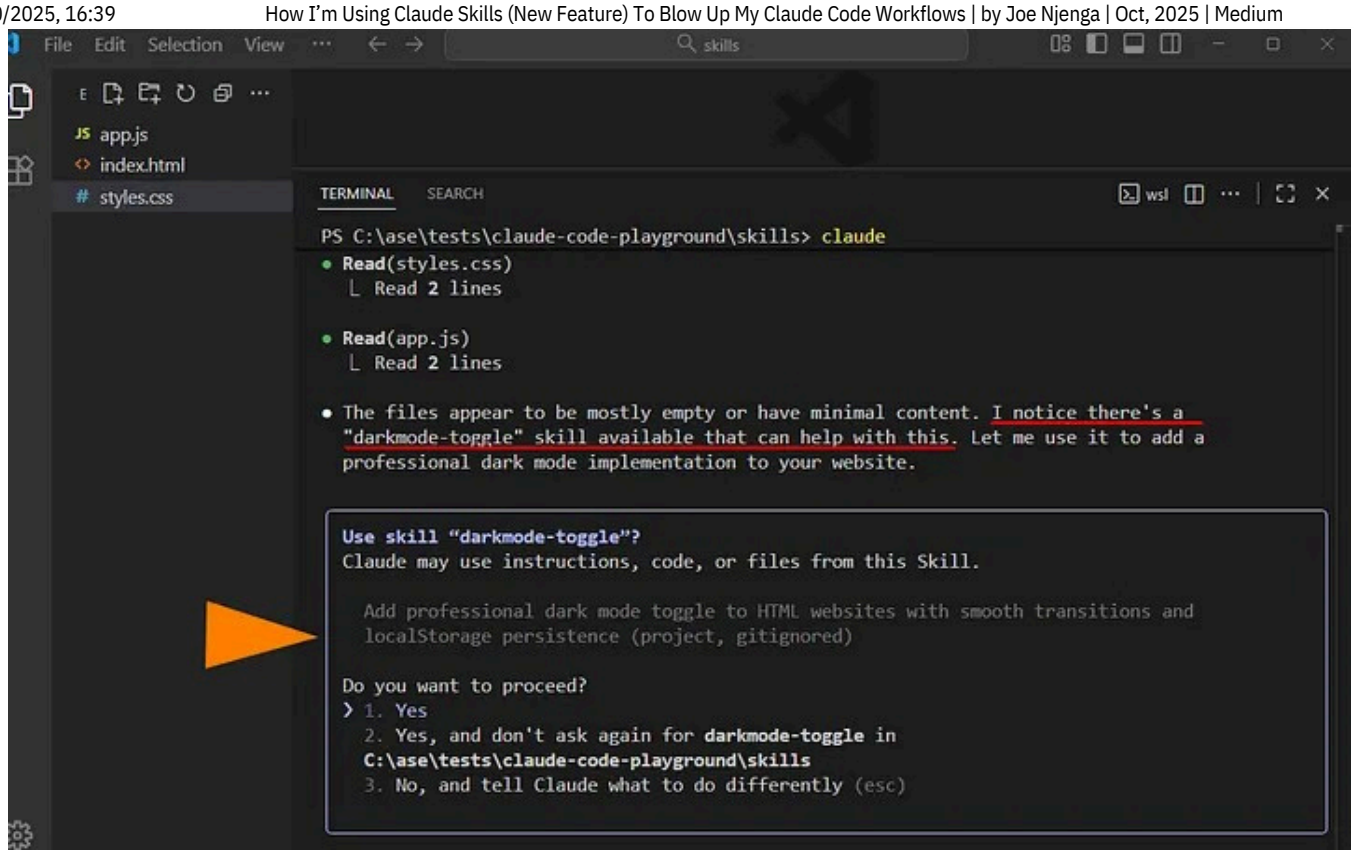


When I typed "Add dark mode to this website," I didn't tell Claude Code to use my darkmode-toggle skill.

Claude scanned my project files, analyzed my request, and then checked the description field in the YAML frontmatter of all my available skills.

```
---
name: Dark Mode Toggle
description: Add professional dark mode toggle to HTML websites with smooth tra
version: 1.0.0
---
```

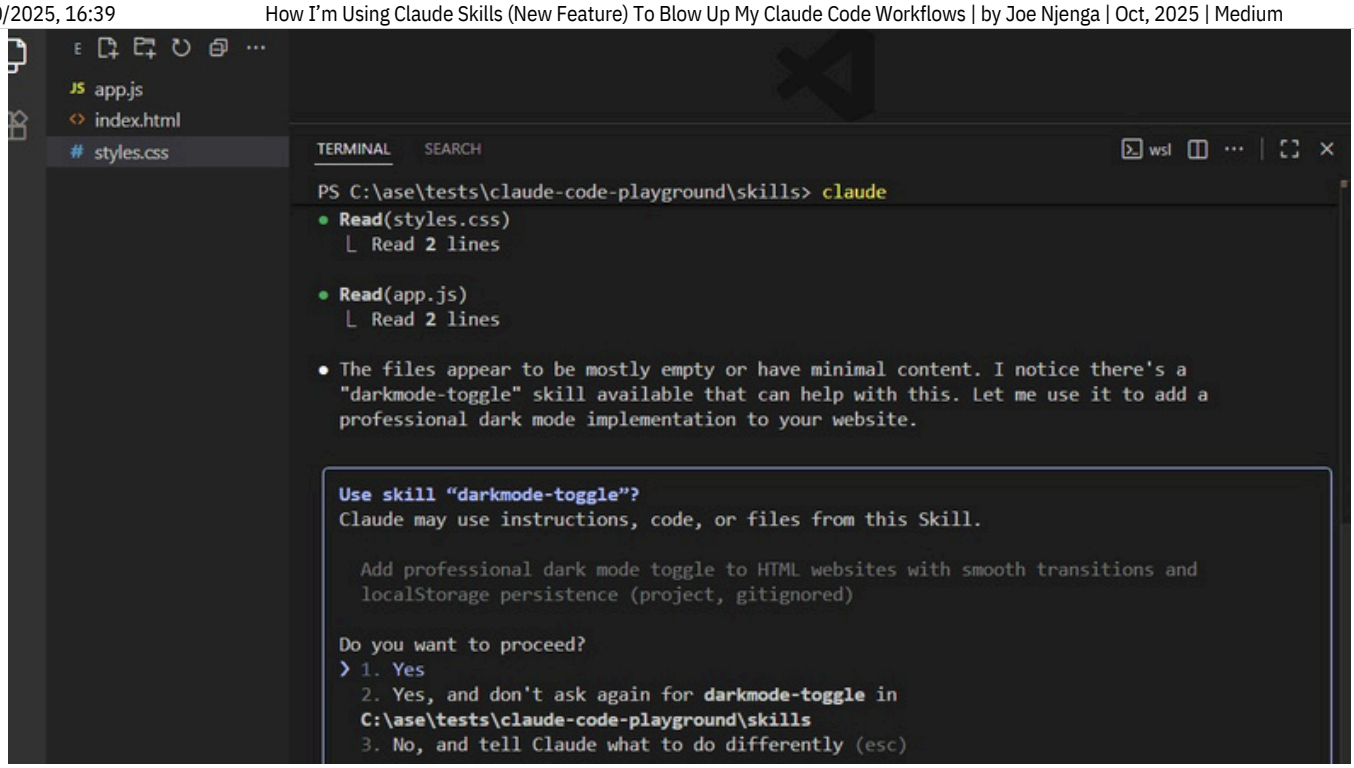
> The description I wrote — "Add professional dark mode toggle to HTML websites with smooth transitions and localStorage persistence" — matched my query .



Claude then prompted me to confirm: “Use skill ‘darkmode-toggle’?”

> This permission step is crucial for security, especially when skills contain executable code. I can choose to approve once, approve for the entire project, or reject and give different instructions.

Claude Skill in Action: Executing My Standards



I clicked "Yes" to approve the skill, and what happened next was remarkable.

> Claude Code immediately started implementing the dark mode feature, but this time, it wasn't making decisions on the fly.

It was following my skill's exact specifications like a blueprint.

Claude updated all three files with:

HTML (index.html) — Added the toggle button exactly as specified:

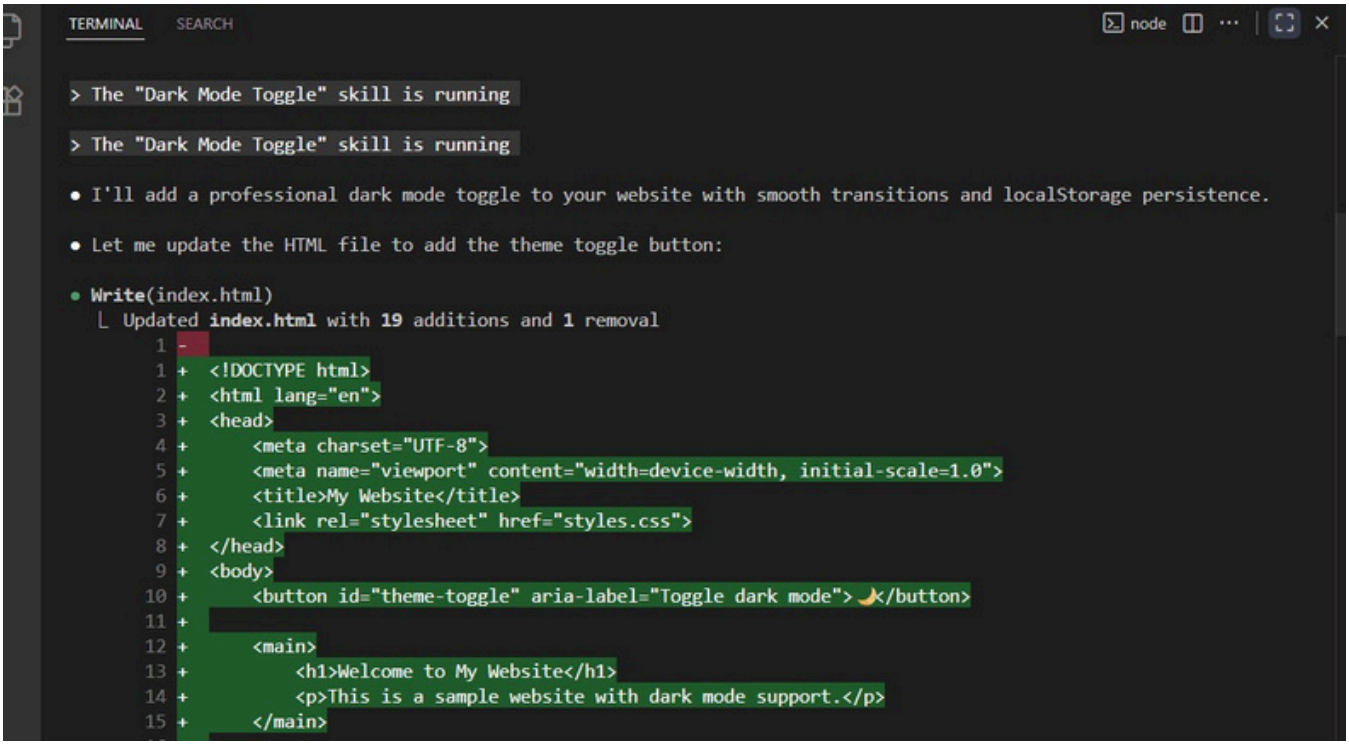
```
<button id="theme-toggle" aria-label="Toggle dark mode">          🌙 </button>
```

Notice the exact ID (theme-toggle), the proper ARIA label for accessibility, and the moon emoji; all matching my skill's pattern.

CSS (styles.css) — Implemented my color scheme and positioning:

```
:root {
--bg-color: #ffffff;
--text-color: #000000;
 --card-bg: #f5f5f5;
}
```

```
[data-theme="dark"] {
--bg-color: #1a1a1a;
--text-color: #ffffff;
 --card-bg: #2d2d2d;
}
#theme-toggle {
 position: fixed;
top: 20px;
right: 20px;
/* ... exact styling from my skill */
}
```



The colors match my specification exactly: #ffffff and #000000 for light mode, #1a1a1a for dark backgrounds.

The button is positioned fixed at top: 20px; right: 20px with the hover effect and transitions I defined.

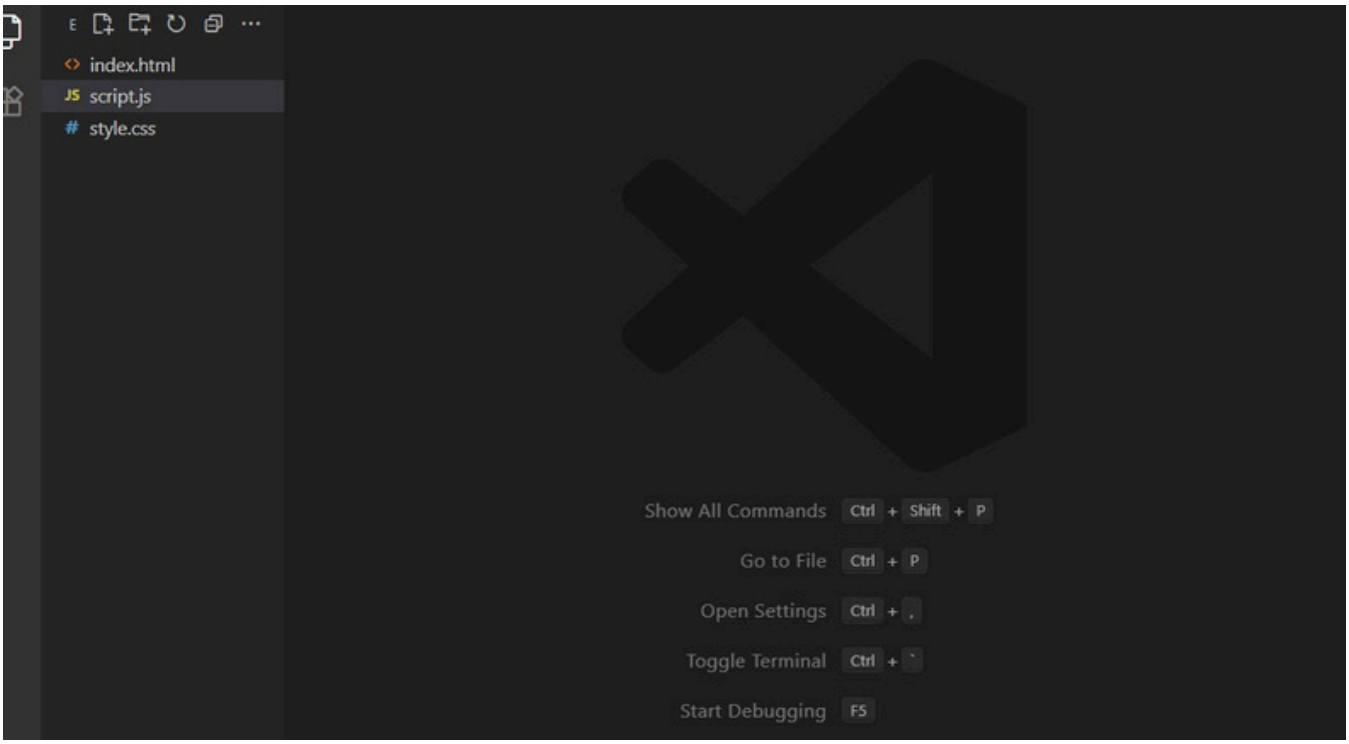
JavaScript (app.js) — My exact localStorage pattern:

```
const themeToggle = document.getElementById('theme-toggle');
const currentTheme = localStorage.getItem('theme') || 'light';
```

```
document.documentElement.setAttribute('data-theme', currentTheme);
themeToggle.textContent = currentTheme === 'light' ? '🌙' : '☀️';
```

Notice it's using document.documentElement.setAttribute('data-theme', ...) As I specified in my skills, not body.classList.add('dark-mode') any other approach.

> This is my standard, and Claude followed it — that's how Claude Skills works with Claude Code to standardize your workflow.



The final output wasn't just "a dark mode implementation."

It was my dark mode implementation, following every detail I had documented in the Skill:

- Exact color values from my style guide
- Specific positioning requirements
- My preferred DOM manipulation method
- The accessibility standards I defined
- localStorage key naming convention
- Smooth 0.3s transitions

> Claude Skills allows you to document your standard once in Claude Code workflows, and now every time you or anyone on your team asks Claude Code for a feature, it implements it your way.

Skill Composition

Here's where it gets even better.

Let's say I also have these Skills in my library:

- responsive-navbar - My standard navigation implementation
- form-validation - Client-side validation with my preferred error styling
- api-integration - My team's fetch wrapper with error handling

Now I can tell Claude Code:

```
"Set up a landing page with a navbar, contact form, and dark mode"
```

> Claude identifies that it needs THREE skills, loads all of them, and composes them together .

Final Thoughts

Now, every time I refine a process or discover a better way to do something, I create the Claude Skill ready to use on Claude Code.

> If you're still manually re-explaining your patterns to Claude every time you start a project, you're working too hard.

You should instead start building your own Skills library that helps eliminate the repetitive tasks in your Claude Code workflow.

If you are ready to build your first Skill;

- Identify your most repeated task — What do you explain to Claude over and over?